Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Разработка программного обеспечения для мобильных платформ

ОТЧЕТ

по лабораторной работе №4

Каталог видеоигр и игровых студий с рейтингами

Выполнил
Студент гр. 351002 Михальков А. Д.

Проверил Петровская В. В.

Минск 2026

СОДЕРЖАНИЕ

1 Введение 3
1.1 Функциональные изменения 3
1.1.1 Облачная аутентификация и управление профилем 3
1.1.2 Реактивное обновление данных 4
1.1.3 Использование аппаратных ресурсов 4
1.1.4 Интеграция с социальными сервисами 5
1.2 Теоретическая основа 6
1.3 Технологии 7
2 Скриншоты работы приложения 8
2.1 Экран регистрации и входа через Firebase 8
2.2 Обновление профиля пользователя 9
2.3 Демонстрация биометрической аутентификации 10
2.4 Отображение геолокации в деталях игры 10
2.5 Демонстрация функции «Поделиться» 11
3 Заключение 13

1 Введение

Цель работы: Расширение функциональных возможностей мобильного приложения путем интеграции системы облачной аутентификации, реализации механизмов реактивного обновления данных в реальном времени, а также использования аппаратных ресурсов мобильного устройства и инструментов социального взаимодействия.

В рамках данной лабораторной работы решаются следующие задачи:

1. Системная аутентификация: Миграция с локального хранения учетных записей на сервис Firebase Authentication. Это обеспечивает высокий уровень безопасности паролей и возможность авторизации пользователя на различных устройствах с сохранением персональных настроек профиля.
2. Реактивная синхронизация данных: Переход от модели разовых запросов к системе постоянной подписки на изменения в удаленной базе данных Firestore. Данная технология позволяет интерфейсу мгновенно отражать любые изменения в контенте (добавление, удаление, редактирование) без необходимости перезагрузки экранов.
3. Взаимодействие с Platform API: Расширение функционала приложения за счет использования нативных ресурсов устройства.
4. Социальная интеграция: Реализация функции нативной передачи данных (Share API), позволяющей пользователю экспортировать ключевую информацию об играх из приложения в сторонние социальные сети и мессенджеры.

1.1 Функциональные изменения

1.1.1 Облачная аутентификация и управление профилем

В рамках разработки была произведена полная замена локальной системы хранения учетных записей на облачный сервис Firebase Authentication. Это позволило перейти от изолированной базы данных пользователей внутри устройства к глобальной системе идентификации.

Взаимодействие с сервисом реализовано через Firebase SDK. Основным идентификатором пользователя теперь является уникальный строковый ключ – UID, генерируемый сервером Google. Для регистрации используется метод, который автоматически проверяет уникальность Email и сложность пароля (минимум 6 символов). Авторизация выполняется методом, возвращающим токен сессии.

Для предотвращения необходимости повторного ввода данных при каждом запуске приложения внедрен слушатель. Он интегрирован в глобальный AuthContext и в реальном времени отслеживает валидность

токена. Если сессия активна, приложение автоматически перенаправляет пользователя на главный экран, минуя окно авторизации.

В разделе настроек был расширен функционал работы с персональными данными. Используя метод updateProfile, пользователь может изменять свой никнейм (displayName), данные о котором сохраняются непосредственно в облачном профиле Firebase. Логика интерфейса настроена на приоритетное отображение установленного никнейма. В случае его отсутствия система автоматически использует часть Email-адреса до символа «@». Смена ника синхронизирована с глобальным состоянием: при успешном ответе от Firebase Auth данные в AuthContext обновляются мгновенно, что влечет за собой автоматическую перерисовку всех связанных компонентов.

Смена системы идентификации потребовала обновления структуры базы данных SQLite: поле owner_id в таблицах игр и истории уведомлений было переведено из целочисленного типа в строковый для корректной привязки контента к UID пользователя.

1.1.2 Реактивное обновление данных

Ключевым изменением стал переход от модели разовых сетевых запросов («запрос – ответ») к системе реактивного наблюдения за состоянием базы данных. Это позволило реализовать мгновенную синхронизацию интерфейса с облачным хранилищем без участия пользователя.

Для реализации данного функционала метод получения документов getDocs был заменен на слушатель onSnapshot из SDK Firebase Firestore. Данный метод устанавливает постоянное соединение с сервером по протоколу WebSocket.

При любом изменении данных в облачной коллекции games (добавление, редактирование или удаление документа) сервер автоматически инициирует отправку нового «снимка» (Snapshot) данных на мобильное устройство.

Функция-слушатель инкапсулирована в useEffect внутри HomeViewModel, что позволяет активировать подписку в момент фокуса на экране и корректно разрывать соединение (unsubscribe) при уходе с экрана или выходе из аккаунта для экономии ресурсов устройства.

Локальная база данных обновляется только в момент срабатывания слушателя onSnapshot.

1.1.3 Использование аппаратных ресурсов

В рамках расширения взаимодействия приложения с аппаратной частью мобильного устройства были интегрированы модули для работы с системой биометрической защиты и спутниковой навигацией.

Для защиты персональных данных пользователя внедрена система «Биометрического замка» на базе библиотеки expo-local-authentication.

Перед активацией функции во ViewModel вызываются методы hasHardwareAsync и isEnrolledAsync для проверки наличия физического сканера (отпечатка пальца или FaceID) и наличия зарегистрированных данных в системе. Если устройство не поддерживает биометрию, соответствующий пункт настроек скрывается из интерфейса.

При включении данной настройки идентификатор состояния сохраняется в энергонезависимую память AsyncStorage. При каждом холодном запуске приложения в файле App.js инициируется вызов системного окна аутентификации. До момента успешного подтверждения личности доступ к основному навигатору приложения заблокирован, отображается специализированный защитный экран.

Для фиксации места добавления игровых записей в каталог интегрирован модуль expo-location.

В приложении реализован алгоритм динамического запроса прав на доступ к службам определения местоположения в момент сохранения формы.

Система считывает текущие GPS-координаты устройства и выполняет операцию «обратного геокодирования» (Reverse Geocoding). С помощью метода reverseGeocodeAsync широта и долгота преобразуются в человекочитаемую строку, содержащую название города и страны (например, «Minsk, Belarus»). Полученная строка записывается в поле location в базах данных SQLite и Firestore.

Доступ к аппаратным функциям осуществляется по запросу, что исключает фоновое потребление заряда батареи. Все вызовы Platform API реализованы асинхронно, предотвращая блокировку основного потока интерфейса (UI Thread) во время ожидания ответа от датчиков устройства.

1.1.4 Интеграция с социальными сервисами

Основной целью данного модуля является интеграция приложения в экосистему социальных сервисов мобильного устройства, позволяющая пользователю транслировать информацию из каталога в сторонние приложения.

Для реализации обмена контентом использован стандартный модуль Share из библиотеки react-native. Данный выбор обусловлен необходимостью использования ресурсов Platform API для вызова системного окна (Share Sheet) операционной системы.

В отличие от жесткой привязки к конкретным SDK (например, Facebook или Telegram), выбранный метод обеспечивает универсальность: пользователь может отправить данные в любую социальную сеть или мессенджер, установленный на устройстве.

Функция вызова окна шеринга инкапсулирована во ViewModel экрана деталей игры (useDetailsViewModel), что позволяет разделять процесс подготовки данных и их визуальное представление.

Система не просто передает ссылку, а генерирует структурированное текстовое сообщение на основе данных из базы данных в реальном времени:

1. При нажатии на кнопку «Поделиться» код собирает значения полей конкретного объекта: полное название игры, студию-разработчика и числовой рейтинг.
2. В сообщение также включается информация о местоположении, полученная через Platform API (Геолокация), если данное поле было заполнено при создании записи.
3. Текстовые метки внутри сообщения (например, «Рейтинг:») подставляются из текущего активного файла перевода (RU/EN), что сохраняет консистентность интерфейса.

1.2 Теоретическая основа

Теоретический базис сосредоточен на принципах облачной идентификации, технологиях передачи данных в реальном времени и механизмах взаимодействия программного обеспечения с аппаратными модулями защиты и навигации.

В отличие от локального хранения учетных данных, облачная аутентификация базируется на использовании уникальных строковых идентификаторов – UID (Unique Identifier). UID является персистентным ключом пользователя в рамках всей экосистемы Google Firebase. Привязка контента к UID вместо инкрементных ID базы данных обеспечивает целостность связей при работе приложения на нескольких устройствах одновременно. Безопасность обеспечивается за счет делегирования обработки паролей на сторону сервера. Приложение оперирует токенами доступа, что исключает хранение чувствительных данных в открытом виде в памяти устройства.

Для реализации обновлений без перезагрузки страниц (Real-time) используется паттерн Observer (Наблюдатель). Механизм onSnapshot в Firebase Firestore основан на использовании протокола WebSocket, который, в отличие от стандартного HTTP-запроса, поддерживает постоянное двустороннее соединение между клиентом и сервером. Это позволяет реализовать событийно-ориентированную архитектуру: сервер инициирует передачу данных сразу после изменения состояния коллекции («Push-модель»), что минимизирует задержки и снижает нагрузку на сетевой трафик по сравнению с методом периодического опроса (Polling).

Работа с Platform API (биометрия и GPS) в современных мобильных ОС (Android/iOS) строится на принципах динамических разрешений (Runtime

Permissions). Программный слой приложения не получает доступа к физическому отпечатку пальца или биометрическому хэшу лица. Взаимодействие происходит через уровень абстракции ОС, которая возвращает приложению только результат сравнения. Это гарантирует высокий уровень приватности. Теоретической основой определения местоположения является сбор данных от спутниковых систем. Процесс реверсивного геокодирования заключается в сопоставлении полученных координат с географическими базами данных для получения адреса (города, улицы) в текстовом формате.

Интеграция с социальными сетями через Share API базируется на использовании системного менеджера. Приложение не реализует логику конкретной соцсети, а передает структурированный массив данных (строки, ссылки) операционной системе, которая, в свою очередь, предлагает пользователю список доступных обработчиков данных (мессенджеров, почтовых клиентов), установленных в системе.

1.3 Технологии

Для реализации расширенного функционала был использован программный стек, обеспечивающий глубокую интеграцию с облачными сервисами и аппаратными ресурсами мобильных платформ:

1. Firebase Authentication: Облачный сервис, обеспечивающий управление учетными записями пользователей.
2. Firestore Real-time SDK (onSnapshot): Программный интерфейс базы данных, реализующий технологию потоковой передачи данных.
3. Expo Location: Библиотека для взаимодействия со службами геолокации устройства. Предоставляет методы для получения текущих GPS-координат и инструменты для реверсивного геокодирования (преобразования координат в физический адрес).
4. Expo Local Authentication: Модуль доступа к средствам биометрической идентификации устройства. Обеспечивает работу со сканерами отпечатков пальцев и системами распознавания лиц (FaceID/TouchID) через стандартные системные диалоги безопасности.
5. React Native Share API: Нативный программный интерфейс, предназначенный для взаимодействия с системным менеджером ресурсов операционной системы. Позволяет передавать текстовые и медиа-данные в любые внешние приложения и социальные сети.

2 Скриншоты работы приложения

2.1 Экран регистрации и входа через Firebase

На Рисунках 1, 2 продемонстрирована работа системы облачной авторизации на базе сервиса Firebase Authentication. Интерфейс обеспечивает регистрацию и вход пользователей по Email-адресу, полностью заменяя локальное хранение учетных данных. Реализованная логика позволяет безопасно идентифицировать пользователя и автоматически синхронизировать его персональную библиотеку игр между различными мобильными устройствами.

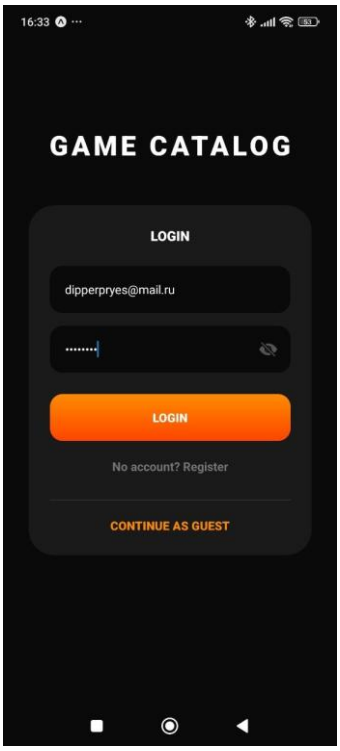

Рисунок 1. Экран авторизации

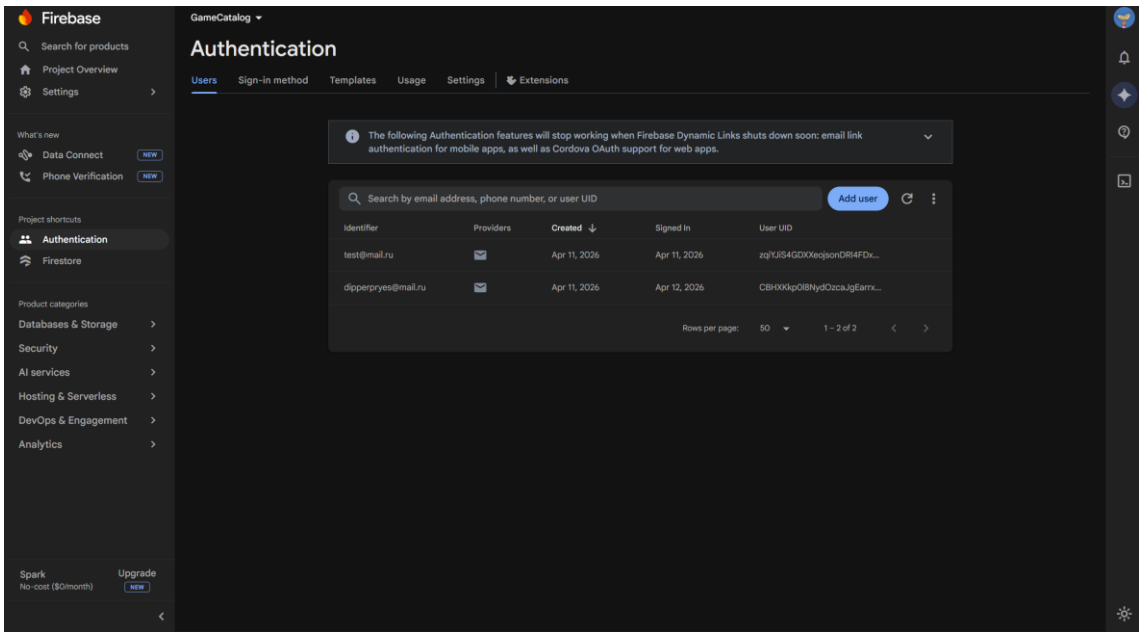

Рисунок 2. Облачное хранилище Firebase

2.2 Обновление профиля пользователя

На Рисунке 3 продемонстрирован интерфейс управления профилем в настройках приложения. Реализована функция смены никнейма пользователя через метод updateProfile сервиса Firebase Auth, что позволяет синхронизировать отображаемое имя между всеми устройствами. Изменения вступают в силу мгновенно и отображаются в заголовке профиля без перезагрузки экрана.

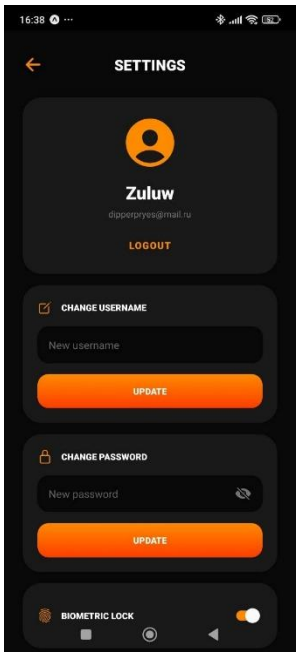

Рисунок 3. Экран настроек

2.3 Демонстрация биометрической аутентификации

На Рисунке 4 представлен защитный экран, блокирующий доступ к приложению при включенной функции биометрического замка. Использование модуля LocalAuthentication позволяет задействовать нативные механизмы защиты устройства (отпечаток пальца или распознавание лица). Доступ к контенту предоставляется только после успешного подтверждения личности через системный диалог ОС.

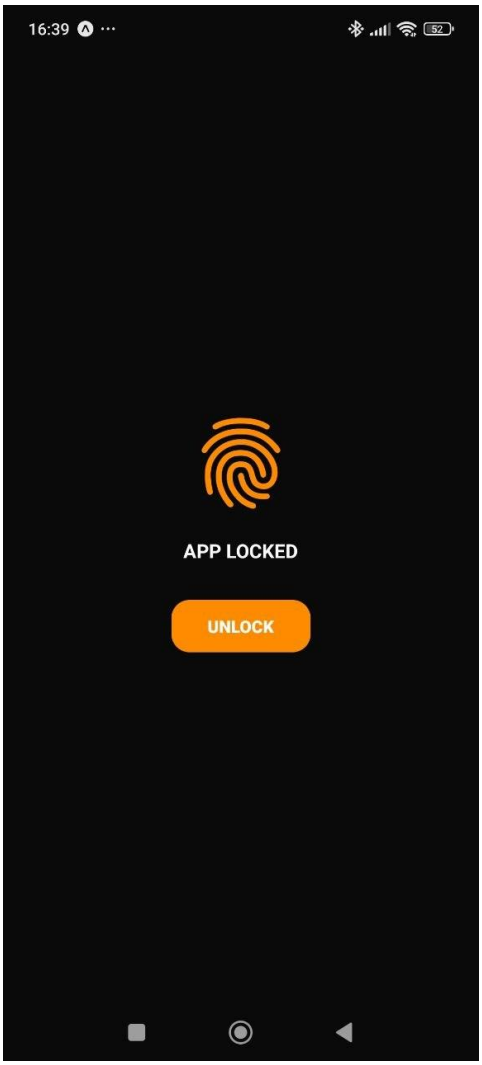

Рисунок 4. Защитный экран

2.4 Отображение геолокации в деталях игры

На Рисунке 5 показан экран просмотра деталей с интегрированной информацией о местоположении. С помощью Platform API (модуль Location) при создании записи были получены координаты устройства, которые автоматически преобразованы в название города и страны. Данная

информация сохранена в облаке и отображается в виде отдельного информационного блока с иконкой геопозиции.

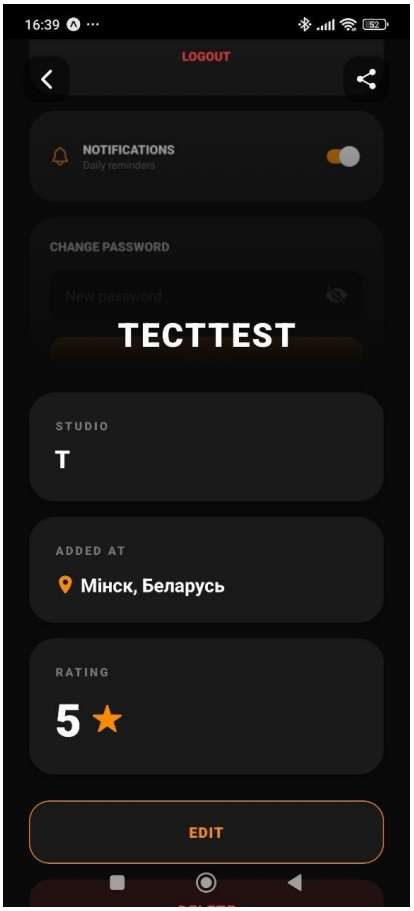

Рисунок 5. Экран просмотра деталей

2.5 Демонстрация функции «Поделиться»

На Рисунках 6, 7 отображена работа функции «Поделиться», реализованной через Native Share API. Приложение генерирует структурированное сообщение, содержащее название игры, рейтинг и местоположение, и передает его системному менеджеру ресурсов. Это позволяет пользователю экспортировать данные в любую социальную сеть или мессенджер, установленный на устройстве.

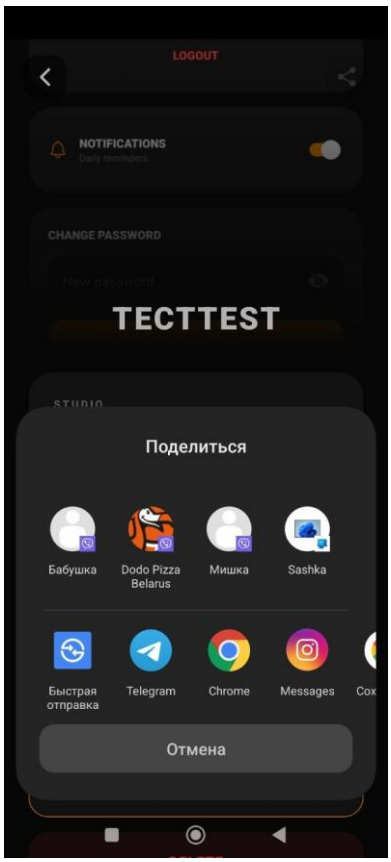

Рисунок 6. Результат работы «Поделиться»

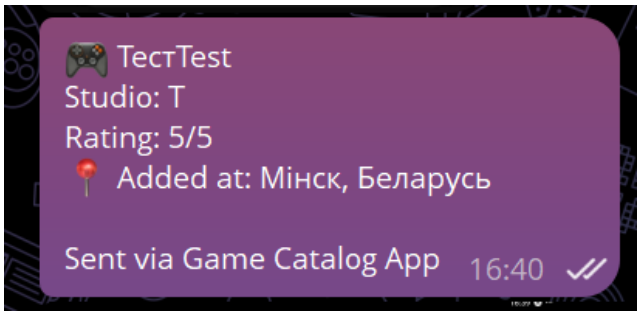

Рисунок 7. Пример структурированного сообщения

3 Заключение

В ходе выполнения работы была успешна завершена разработка мобильного приложения.

Итогом данного этапа стала глубокая интеграция приложения с внешними сервисами идентификации и аппаратными ресурсами мобильной платформы. Внедрение системы Firebase Authentication позволило реализовать безопасную модель аутентификации пользователей и перейти к управлению глобальными профилями, что обеспечило доступность персонального контента на различных устройствах под единой учетной записью.

Важным достижением стал переход на реактивную модель обновления данных через механизмы прослушивания снимков базы данных Firestore в реальном времени. Это позволило полностью устранить необходимость ручной перезагрузки интерфейса и обеспечило мгновенную синхронизацию изменений между всеми клиентами системы.

Особое внимание в работе было уделено использованию Platform API, что позволило задействовать аппаратные модули устройства для повышения безопасности и информативности контента. Реализованный биометрический замок на базе FaceID и отпечатков пальцев обеспечил надежную защиту персональных данных, а интеграция служб геолокации позволила автоматически дополнять записи информацией о месте их создания с использованием технологий реверсивного геокодирования.

Социальная интеграция через Native Share API превратила приложение в открытую систему, позволяющую пользователю эффективно транслировать контент в сторонние мессенджеры и социальные сети.

Все реализованные функции были вписаны в архитектурный паттерн MVVM, что продемонстрировало высокую гибкость и масштабируемость спроектированной системы.

Завершение работы подтвердило готовность приложения к эксплуатации в качестве полноценного мобильного решения, соответствующего современным стандартам безопасности, производительности и качества пользовательского опыта.

Ссылка на репозиторий с итоговым кодом:
https://github.com/zuluw/game-catalog